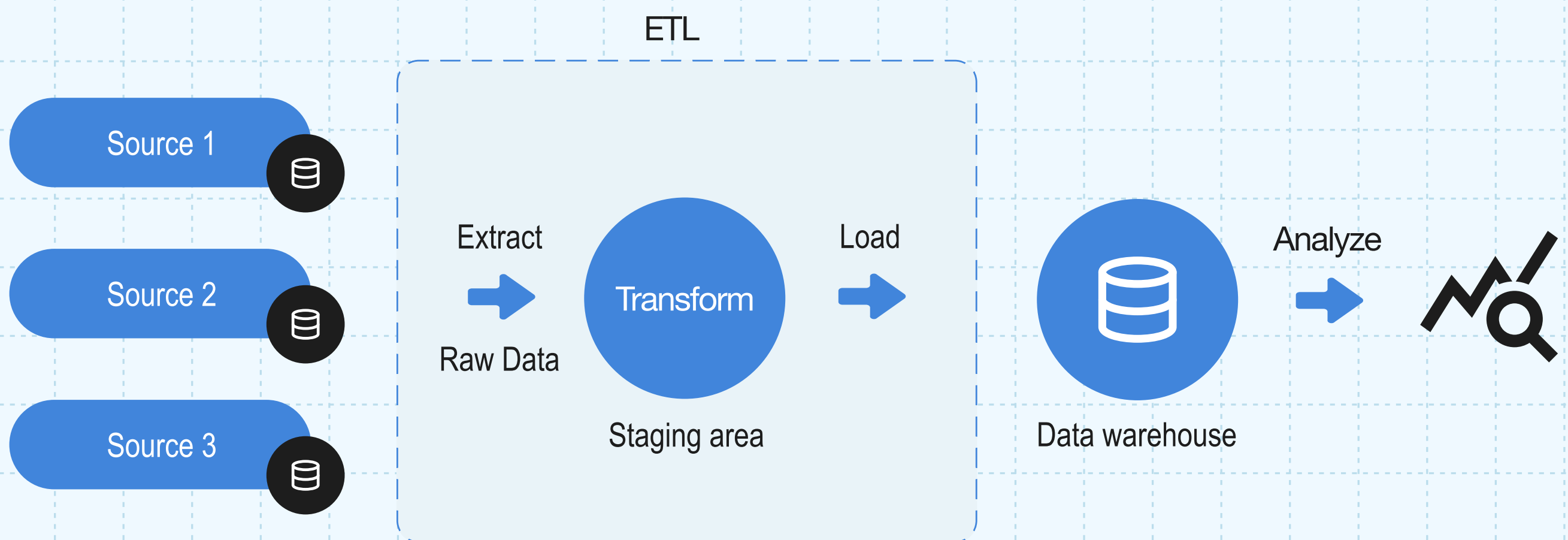


UNDERSTANDING ETL PIPELINE FOR DATA ENGINEERS





Disclaimer

EVERYONE LEARNS UNIQUELY.

This roadmap is a guide to help you navigate the journey of becoming a Data Engineer.

Treat this as direction, not a fixed rulebook. Real growth comes from consistent practice and solving real-world problems.

TABLE OF CONTENTS

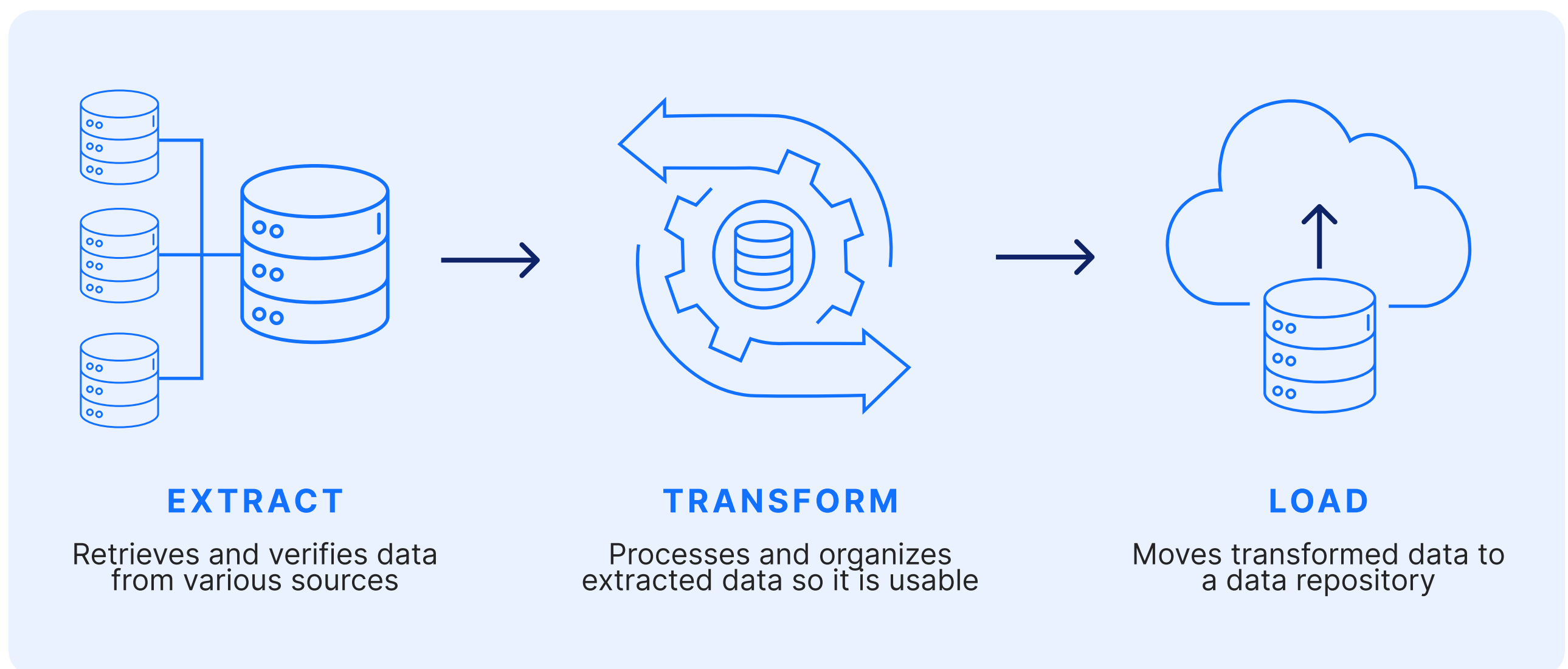
01	INTRODUCTION	4-9
	What Is ETL And Why It Matters	
	Importance For Analytics, Machine Learning, And Operations.	
	ETL Vs ELT Explained Simply.	
02	ETL BASICS	10-16
	Extract → Gather Data From Sources	
	Transform → Clean, Standardize, Enrich	
	Load → Store Into Warehouse/Lake	
	Batch Vs Real-Time Pipelines + Challenges	
03	ETL ARCHITECTURE	17-21
	Layers: Sources → Staging → Transformation → Warehouse → Consumption	
	Shows The Structured Flow From Raw Data To Insights	
04	TOOLS & TECHNOLOGIES	22-31
	Ingestion: Kafka, Fivetran, Glue	
	Transformation: Spark, Dbt, Pandas	
	Storage: Snowflake, BigQuery, S3	
	Orchestration: Airflow, Prefect	
	Monitoring: Great Expectations, Monte Carlo	
05	BEST PRACTICES	32-33
	Modular Design, ELT In Modern Warehouses	
	Automated Data Quality Checks	
	Schema Evolution, Monitoring, Scalability	
	Shows Real Business Value Of ETL Pipelines	
06	USE CASES	34
	Daily Sales Reporting → Walmart	
	Customer Personalization → Amazon	
	Inventory Optimization → Target	
	Shows Real Business Value Of ETL Pipelines	

Introduction

01 What Is ETL In Data Engineering?

- **ETL (Extract, Transform, Load)** is the process of moving raw data from different sources into a centralized storage system.
- **Extract** - Data is collected from multiple sources such as databases, APIs, logs, IoT devices, or streaming platforms.
- **Transform** - The raw data is cleaned, standardized, enriched, and reshaped to match business requirements.
- **Load** - The processed data is stored in a target system like a data warehouse (Snowflake, Redshift, BigQuery) or a data lake (S3, GCS, HDFS).
- For Data Engineers, ETL pipelines are critical to ensure that data is **reliable, scalable, and accessible** for analysts, data scientists, and business teams.

THE ETL PROCESS EXPLAINED



ETL pipelines are the **backbone of data-driven organizations**. Their importance lies in:

1. Analytics & Business Intelligence (BI):

- Structured, aggregated data supports dashboards and reports.
- Leadership can make better decisions based on accurate, timely insights.

2. Machine Learning & Data Science:

- High-quality data enables effective feature engineering.
- Models perform better when trained on clean, consistent datasets.

3. Operational Efficiency:

- Automates repetitive data preparation tasks.
- Reduces manual effort and ensures data freshness.

4. Data Consistency:

- Establishes a single source of truth, avoiding conflicts across teams.
- Maintains data governance and compliance.

ETL pipelines are the **backbone of data-driven organizations**. Their importance lies in:

1. Traditional ETL (Extract → Transform → Load):

- Data is transformed before loading into the warehouse.

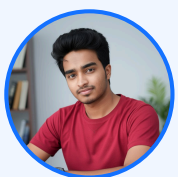
- Requires external ETL tools or processing engines.
- Was efficient when compute and storage were **expensive and limited**.

2. Modern ELT (Extract → Load → Transform):

- Raw data is loaded **first** into the warehouse, and transformations are performed inside it.
- Cloud-native systems like Snowflake, BigQuery, and Redshift make this approach faster and more scalable.
- Simplifies architecture by centralizing transformations in the warehouse.

3. Key Difference:

- ETL - Transformation happens outside the warehouse.
- ELT - Transformation happens inside the warehouse after loading.



Ajay Naik



My experience with Bosscoder was **life-changing**. The program gave me **valuable mentorship, resume optimization, and structured assignments**. The teaching was excellent, and the supportive team boosted my confidence. It turned out to be a **great investment for career growth**.

What is ETL?

01 Breakdown Of Extract → Transform → Load

1. Extract

- In this step, data is pulled from multiple sources such as relational databases, NoSQL systems, APIs, log files, and streaming platforms like Kafka.
- The goal is to gather all relevant data, whether structured, semi-structured, or unstructured, in its raw form.
- A key challenge during extraction is ensuring that the data is collected completely and accurately without any loss.

2. Transform

- Transformation is the process of converting raw data into a clean, consistent, and usable format.
- Typical operations include cleaning the data (removing duplicates and handling nulls), standardizing formats (data types, units, and structures), and aggregating information (summing sales, calculating averages, etc.).
- It can also involve enriching data by joining it with other datasets or creating new derived fields.
- This step ensures the data is aligned with business logic and ready for analysis.

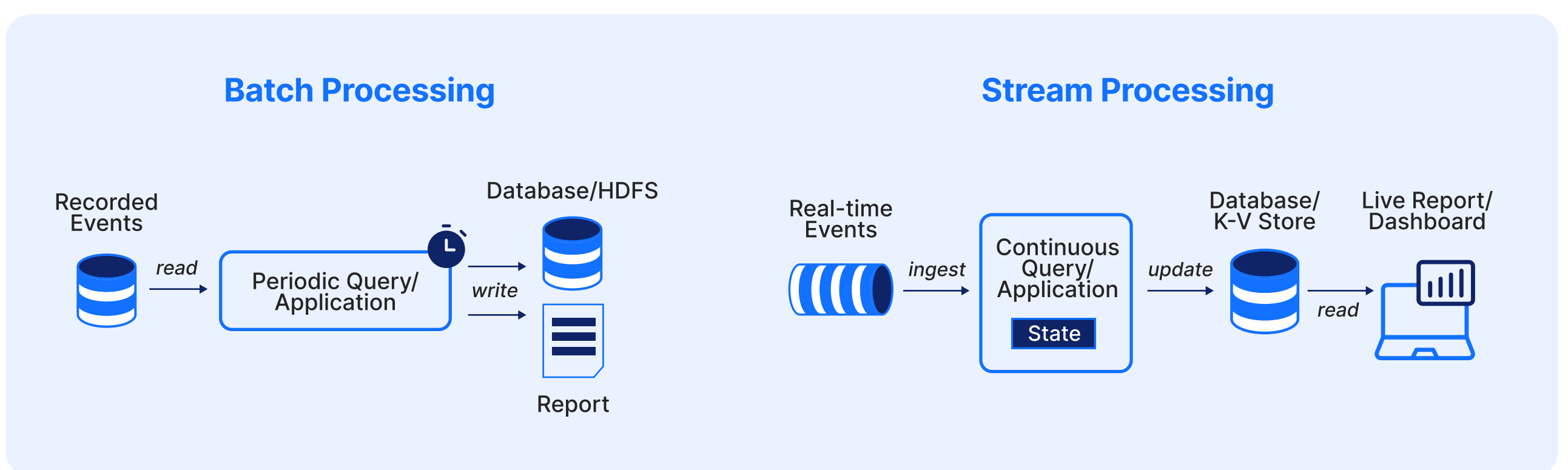
3. Load

- The final step involves loading the transformed data into a destination system such as a data warehouse (Snowflake, BigQuery, Redshift), a data lake (S3, GCS, HDFS), or operational databases.

- Loading can be done in two ways:
 - a. Full load**, where all the data is reloaded each time.
 - b. Incremental load**, where only new or updated records are loaded.
- The end goal is to make the data easily accessible for BI tools, reporting, or machine learning applications.

02

Batch Vs Real-Time Pipelines



1. Batch ETL

- Batch pipelines process large amounts of data at scheduled intervals such as hourly, daily, or weekly.
- This approach is commonly used for periodic reporting and historical analysis where real-time data is not critical.
- Tools such as Apache Airflow, Apache Spark (batch mode), and AWS Glue are often used to build batch pipelines.

Example: A daily sales report that processes all transactions at midnight.

2. Real-Time (Streaming) ETL

- Real-time pipelines process and deliver data continuously as it is generated.

- This method is essential for time-sensitive use cases such as fraud detection, stock trading, or personalized recommendations.

3. Trade-off

- Batch pipelines are simpler to build and more cost-effective, but they do not provide the most up-to-date data.
- Real-time pipelines deliver fresh insights with low latency but are more complex and expensive to maintain.

03 Common Challenges In ETL

1. Handling Large Data Volumes

- Modern organizations generate terabytes or even petabytes of data, which requires highly scalable ETL solutions.
- Distributed processing frameworks like Apache Spark and Apache Flink are often needed to manage this scale efficiently.

2. Schema Drift

- Over time, data sources often evolve by adding new fields, changing data types, or modifying formats.
- ETL pipelines must be designed to handle these schema changes gracefully without breaking.

3. Data Quality Issues

- Inconsistent, missing, or duplicate records can reduce the reliability of insights.
- Data quality checks and validation rules are necessary to ensure trust in the processed data.

4. Latency and Performance

- Balancing the need for fast data availability with system costs can be difficult.
- Real-time pipelines provide low latency but require significant infrastructure, while batch pipelines are slower but cheaper.

5. Reliability and Fault Tolerance

- ETL pipelines must be resilient and capable of recovering from failures without data loss.
- Monitoring, logging, and retry mechanisms are critical for maintaining reliability.

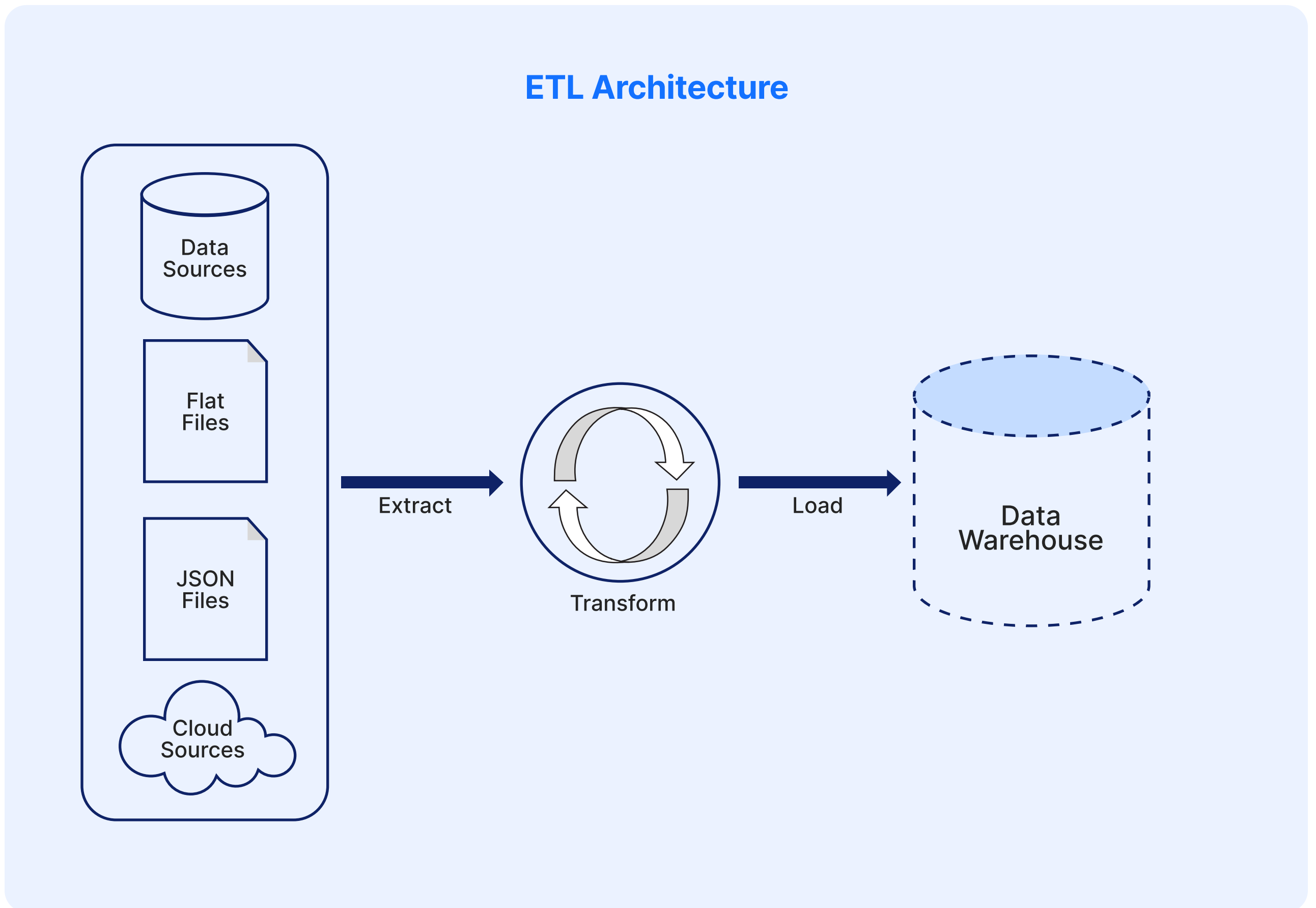


Prince Yadav



I struggled to find a structured way to upskill until I joined Bosscoder. The **live sessions** were detailed and the mentorship was highly impactful. **Consistent learning, mock interviews, and personal guidance** helped me grow in clarity and confidence. Bosscoder proved to be the right platform for my career transition.

ETL Architecture



01 Data Sources (Input Layer)

- **What it is:** The entry point of data into the ETL pipeline.

Example:

- a. **Databases:** MySQL, PostgreSQL, MongoDB.
- b. **APIs:** Product catalogs, social media APIs.

- c. **Logs:** Web server logs, application logs.
- d. **Streaming:** Kafka, AWS Kinesis, IoT devices.

- **Purpose:** Gather raw data in **various formats (structured, semi-structured, unstructured)**.
- **Key Point:** The extraction must ensure **accuracy and completeness** without affecting the performance of the source systems.

02 Staging Area (Raw Data Layer)

- **What it is:** A temporary storage layer where raw data lands before any processing.

Example: Storing JSON/CSV files in Amazon S3, Google Cloud Storage, or HDFS.

- **Purpose:**
 - a. Acts as a buffer to decouple source systems from the pipeline.
 - b. Provides a safe copy of raw data for reprocessing if needed.
- **Key Point:** Ensures that even if transformation fails, raw data is preserved.

03 Transformation Layer (Processing)

- **What it is:** The heart of ETL where raw data is cleaned, standardized, and enriched.
- **Operations:**
 - a. **Cleaning:** Removing duplicates, handling null values.
 - b. **Standardization:** Converting data types, currencies, or time zones.

- **Aggregation:** Calculating daily sales totals, averages, KPIs.
- **Enrichment:** Joining with external datasets like demographics or product metadata.
- **Business Rules:** Applying logic like flagging “high-value customers.”
- **Tools:** Apache Spark, dbt, Pandas, SQL scripts, Azure Data Factory.
- **Key Point:** Ensures data is **consistent, reliable, and analysis-ready**.

04 Warehouse / Data Lake (Destination Layer)

- **What it is:** The final storage system where transformed data is kept for consumption.
- **Options:**
 - a. **Data Warehouses:** Snowflake, BigQuery, Redshift (for structured, analytics-ready data).
 - b. **Data Lakes:** S3, GCS, Delta Lake (for raw + semi-structured storage).
- **Loading Approaches:**
 - a. **Batch Load:** Large data chunks at scheduled times.
 - b. **Streaming Load:** Continuous updates for real-time analytics.
- **Key Point:** This layer becomes the **single source of truth** for the organization.

05 Consumption Layer (Analytics / ML / BI)

- **What it is:** The layer where end-users and applications use the data.

Example:

- a. **BI Tools:** Tableau, Power BI, Looker → for dashboards and reports.
 - b. **Machine Learning Models:** Use curated data for predictive analytics.
 - c. **APIs/Apps:** Expose data to business applications or customer-facing systems.
- **Key Point:** This is where business value is created from the ETL pipeline.

WHY THIS ARCHITECTURE MATTERS

- Provides a **structured flow** from raw data to insights.
- Ensures **scalability** as data grows.
- Maintains **data quality and governance**.
- Supports both **batch** and **real-time** use cases.
- Creates a **single source of truth** for decision-making.

Tools & Technologies in ETL Pipelines

ETL pipelines don't rely on a single tool. Instead, they use a **stack of technologies** covering:

- Data ingestion (Extract)
- Processing & transformation (Transform)
- Storage (Load)
- Orchestration (Scheduling & Automation)
- Monitoring & Quality Assurance

01 Data Ingestion Tools

These are used for the **Extract** phase. They pull data from different sources such as databases, APIs, logs, and event streams.

1. Apache Kafka

- A distributed streaming platform.
- Handles real-time ingestion of data like website clickstreams, IoT data, or financial transactions.

Example: Netflix uses Kafka to capture streaming events for real-time recommendations.

2. Fivetran / Stitch

- Fully managed SaaS-based ingestion tools.

- Provide ready-made connectors for Salesforce, Shopify, HubSpot, MySQL, etc.
- Great for businesses that don't want to maintain ingestion infrastructure.

3. AWS Glue / Google Dataflow / Azure Data Factory

- Cloud-native managed services for ETL/ELT.
- Provide built-in connectors, scheduling, and scaling.

Example: AWS Glue can automatically crawl S3 and build ETL jobs without heavy coding.

02

Transformation Tools

These handle the **Transform** phase, where raw data becomes structured, clean, and analysis-ready.

1. Apache Spark / PySpark

- Distributed data processing framework.
- Handles large-scale batch processing and real-time (Spark Streaming).
- Supports SQL, Python, Scala, R, and Java APIs.

Example: Cleaning and aggregating terabytes of e-commerce transaction logs.

2. dbt (Data Build Tool)

- Transformation tool that works inside warehouses.
- Uses SQL to build data models and transformations.

Example: Analysts define data marts in Snowflake using dbt SQL scripts.

3. Pandas / Dask

- Python libraries for small to medium scale data transformations.
- Pandas → single machine.
- Dask → distributed processing similar to Spark.

Example: Cleaning a dataset of 10M rows before loading into ML models.

4. Apache Beam / Apache Flink

- Unified batch and stream processing frameworks.
- Ideal for advanced real-time transformations.

Example: Fraud detection system processing thousands of transactions per second.

03 Storage & Destinations (Load Layer)

The **Load** phase stores processed data for long-term use.

1. Data Warehouses (structured, optimized for analytics):

- **Snowflake:** Fully managed, supports semi-structured data (JSON, Parquet). Auto-scaling.
- **Google BigQuery:** Serverless, pay-per-query, extremely fast for analytics.
- **Amazon Redshift:** Scalable warehouse integrated with AWS ecosystem.
- **Azure Synapse:** Microsoft's warehouse for SQL + big data integration.

2. Data Lakes (raw + semi-structured storage):

- **Amazon S3:** Scalable object storage, often used as a “data lake.”
- **Google Cloud Storage (GCS):** Similar to S3, integrated with GCP.

- **Delta Lake:** Open-source lakehouse framework on top of Spark.
- **HDFS (Hadoop Distributed File System):** Legacy distributed storage system.

3. Lakehouse Platforms (merge lakes + warehouses):

- **Databricks (Delta Lake):** Combines flexibility of data lakes with performance of warehouses.
- **Apache Iceberg / Apache Hudi:** Table formats enabling schema evolution and ACID transactions in lakes.

04 Orchestration Tools

In an ETL pipeline, it's not enough to just extract, transform, and load data — you also need a way to organize, schedule, and monitor these steps so they run reliably.

That's where orchestration tools come in. Think of them as the “**project managers**” of ETL pipelines:

1. **Scheduling:** Decide when jobs run (hourly, daily, real-time).
2. **Automation:** Make sure jobs run automatically without manual effort.
3. **Dependencies:** Ensure steps happen in the right order (you can't transform before extraction).
4. **Monitoring:** Keep track of success/failure, send alerts if something breaks.
5. **Apache Airflow**
 - Most widely used open-source orchestration tool.
 - Defines workflows as DAGs (Directed Acyclic Graphs).

[Example:](#) Schedule “extract sales - transform in Spark - load into Redshift.”

6. Prefect

- Modern alternative to Airflow with easier deployment.
- Cloud-managed or open-source.

[Example:](#) Manage 1,000+ daily ETL tasks with retries and alerts.

7. Dagster

- Orchestration tool with strong focus on **data quality & lineage**.

[Example:](#) Enforce schema validation while scheduling transformations.

8. Luigi

- Lightweight Python-based orchestration.

[Example:](#) Automating smaller ETL jobs like daily CSV ingestion → SQLite load.

9. Cloud-Native Schedulers:

- **AWS Step Functions, GCP Composer (managed Airflow), Azure Data Factory pipelines.**
- Great for teams already tied to specific cloud ecosystems.

05

Monitoring & Data Quality Tools

Pipelines break — monitoring ensures reliability and data trust.

1. Prometheus + Grafana

- Monitor ETL infrastructure (CPU, memory, latency).
- Grafana dashboards show pipeline health.

2. Monte Carlo, Bigeye, Datafold

- Specialized “data observability” platforms.
- Track lineage, anomalies, and ensure data freshness.

3. Great Expectations

- Open-source framework for **data validation**.

Example: Ensure “customer_id” is never null or duplicated.



Gaini Alfred Richards



I joined Bosscoder to upskill and switch to better opportunities. The flexibility of **recorded lectures** and **strong doubt support** made learning easy despite my busy schedule. Even before completing my course, the **structured guidance** helped me transition successfully. Bosscoder continues to support my growth journey.

Best Practices for ETL Pipelines

ETL pipelines are not just technical scripts, they are the **nervous system of data-driven companies**. Designing them well ensures **scalability, trust, and business impact**. Below are the core best practices explained with **theory + real-world use cases**.

01 Modularity And Reusability

- **Principle:** Design ETL pipelines as separate modules (extract, transform, load) rather than one monolithic job.
- **Why it Matters:** Modularity makes it easier to debug, maintain, and reuse components across different datasets.
- **Use Case:**
 - a. **Airbnb** extracts booking data and user activity separately but applies reusable “user ID cleanup” transformations in both pipelines.
 - b. This avoids duplicate code and ensures consistency across datasets.

02 ELT Over ETL In Modern Warehouses

- **Principle:** Load raw data into the warehouse first, then transform it there (ELT).
- **Why it Matters:** Modern warehouses like **Snowflake** and **BigQuery** can scale transformations cheaply and fast, reducing pipeline complexity.

- **Use Case:**
 - a. **Spotify** loads raw streaming logs directly into BigQuery.
 - b. Analysts and dbt transformations then shape it into reporting tables for dashboards without needing external Spark jobs.

03 Automated Data Quality Checks

- **Principle:** Validate data automatically before it reaches end-users.
- **Why it Matters:** Faulty data leads to wrong business decisions and mistrust in the pipeline.
- **Use Case:**
 - a. **Uber** uses automated anomaly detection to check if ride transaction counts suddenly drop.
 - b. If data fails, alerts are triggered before dashboards show incorrect metrics.

04 Schema Evolution And Version Control

- **Principle:** Expect data sources to change (new fields, type changes). Track and version schema definitions.
- **Why it Matters:** Prevents pipelines from breaking when source data evolves.
- **Use Case:**
 - a. **Netflix** maintains schema definitions for their event logs in Git.
 - b. If a new column (like “ad_type”) is added, transformations adapt automatically instead of failing.

05

Monitoring And Observability

- **Principle:** Treat ETL like production software — monitor jobs, latency, and data freshness.
- **Why it Matters:** Detects silent failures quickly and prevents corrupted data from spreading.
- **Use Case:**
 - a. **Amazon** monitors its retail ETL pipelines with dashboards showing data lag.
 - b. If a batch job is delayed, alerts are sent to on-call engineers before sales dashboards break.

06

Performance And Scalability

- **Principle:** Optimize pipelines for incremental loads, partitioning, and distributed compute.
- **Why it Matters:** Prevents slow queries and unnecessary costs as data grows.
- **Use Case:**
 - a. **Netflix** partitions viewing data by region + date in S3.
 - b. Queries for “US viewership in July” only scan relevant partitions, making them 10x faster.

07

Security And Compliance

- **Principle:** Apply encryption, masking, and governance to protect sensitive data.

- **Why it Matters:** Data pipelines often carry **PII (Personally Identifiable Information)** - compliance with GDPR, HIPAA, etc., is mandatory.
- **Use Case:**
 - a. **Stripe** masks customer card numbers before they enter analytics systems.
 - b. Only the last 4 digits are retained for reporting, ensuring compliance.

08 Data Lineage And Transparency

- **Principle:** Track where data came from and how it was transformed.
- **Why it Matters:** Provides auditability, trust, and faster debugging when errors occur.
- **Use Case:**
 - a. **LinkedIn** built DataHub (now open source) to track lineage across all pipelines.
 - b. Analysts can see how a dashboard metric was derived, back to raw source logs.

09 Cost Awareness

- **Principle:** Optimize resource usage by scheduling, archiving, and auto-scaling.
- **Why it Matters:** ETL jobs can rack up high compute/storage bills if not managed.
- **Use Case:**
 - a. **Twitter (X)** schedules non-urgent analytics pipelines at off-peak hours.
 - b. This saves millions annually by avoiding peak cloud compute costs.

Use Case Examples – Retail Analytics with ETL

01 Daily Sales Reporting

Problem

Retailers generate **millions of transactions every day** across physical stores, online platforms, and third-party vendors. Each channel often stores its data separately — POS (Point of Sale) systems for in-store purchases, relational databases for e-commerce, and payment gateways for online transactions.

- This leads to **fragmented, delayed, and inconsistent reporting**.
- Business leaders can't easily answer: "What were today's total sales across all regions and channels?"

ETL Flow

1. Extract:

- Pull transactions from POS systems, relational DBs, and APIs.
- Collect payment gateway logs for online orders.

2. Transform:

- Clean duplicate records caused by multiple system updates.
- Convert different currencies into a single base currency.

- Standardize time zones (e.g., UTC for global comparison).
- Aggregate KPIs such as **daily sales by store, product category, and region**.

3. Load:

- Load into a centralized data warehouse like Snowflake, Redshift, or BigQuery.
- Create dashboards in Tableau or Power BI for leadership.

Business Impact

- Executives see **up-to-date, accurate sales dashboards** every morning.
- Regional managers can identify which stores need **immediate interventions** (e.g., promotions for underperforming products).
- Finance teams can **forecast revenues more accurately** with consolidated data.

Real-World Example

- **Walmart** uses massive ETL pipelines to process data from **11,000+ stores worldwide**, giving leadership near real-time access to sales data. Their system processes **2.5 PB/hour**, ensuring decisions are made on fresh data.

Problem

Modern consumers demand **personalized shopping experiences**. But customer data is often **siloed**: loyalty programs, CRM, website clickstreams, and in-store purchase history are all stored separately.

- Without integration, retailers send **generic promotions** that fail to engage customers.
- This leads to **missed opportunities** for cross-selling, upselling, and retention.

ETL Flow

1. Extract:

- Pull customer data from CRM (profiles, loyalty points).
- Stream clickstream logs from websites and apps via **Kafka**.
- Gather past purchase history from transactional databases.

2. Transform:

- Merge duplicate customer IDs across systems (same customer may appear in CRM + app + loyalty program).
- Clean and standardize demographic data (age, location, income group).
- Enrich with behavioral patterns (frequent shopper vs one-time buyer).
- Segment customers into **loyal, occasional, at-risk, or new shoppers**.

3. Load:

- Store unified customer profiles in a warehouse.
- Feed data into ML models for product recommendations.

Business Impact

- **Personalized recommendations** (“You bought running shoes, here’s a discount on sports socks”).
- **Targeted offers** to high-value or at-risk customers.
- Improved **email/SMS marketing conversion rates**.
- Increased **customer retention and lifetime value (CLV)**.

Real-World Example

- **Amazon** attributes **35% of its total revenue** to its recommendation system, powered by ETL pipelines that unify browsing + purchase data in near real time.

Problem

Inventory management is one of retail's biggest challenges. Stockouts frustrate customers and cause lost sales, while overstocking ties up capital and increases warehousing costs.

- With **inventory data scattered across warehouses, suppliers, and ERP systems**, retailers lack **real-time visibility**.
- This leads to inefficient restocking and poor supply chain decisions.

ETL Flow

1. Extract:

- Pull stock levels from ERP systems and warehouse DBs.
- Fetch supplier availability from external APIs.

2. Transform:

- Normalize units (e.g., cartons, packs, and singles converted into "units").
- Aggregate current stock by location and compare against predicted demand.
- Apply business rules (e.g., mark products with <10% buffer stock as "critical").

3. Load:

- Store into a warehouse + data lake.
- Power dashboards that show **inventory health** by product, store, and region.
- Feed into ML models for **demand forecasting**.

Business Impact

- Prevents **lost sales** by ensuring popular items are always in stock.
- Reduces **warehousing costs** by avoiding excess inventory.
- Improves **supplier coordination** with predictive restocking alerts.

Real-World Example

- **Target** leverages BigQuery + ETL pipelines to track inventory across **1,900+ stores** in real time. Predictive restocking reduces stockouts, improves operational efficiency, and enhances customer satisfaction.

Summary

These three use cases show how ETL pipelines transform raw data into **business value**:

- 1. Daily Sales Reporting:** Unified visibility into revenue.
- 2. Customer Personalization:** Smarter, targeted engagement.
- 3. Inventory Optimization:** Efficient supply chain management.



WHY BOSSCODER?

01

STRUCTURED INDUSTRY-VETTED CURRICULUM

Our curriculum covers everything you need to get become a skilled software engineer & get placed.

02

1:1 MENTORSHIP SESSIONS

You are assigned a personal mentor currently working in Top product based companies.

03

2200+ ALUMNI PLACEMENT

2200+ Alumni placed at Top Product-based companies.

04

24 LPA AVERAGE PACKAGE

Our Average Placement Package is **24 LPA** and highest is **98 LPA**



Niranjan Bagade

Software Engineer,
British Petroleum

10 Years
Experience

NICE

Software Eng.
Specialist

Hike

■ **83%**



British Petroleum

Software Engineer



Dheeraj Barik

Software Engineer 2,
Amazon

2 Years
Experience

Infosys

Software Engineer

Hike

■ **550%**



Amazon

SDE 2

[**EXPLORE MORE**](#)